SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Madhavan. S	**Reg. No.:**	192425113
Section / Batch:	CSE-AI/2024/D	**Date:**	06/08/2026

Code of Conduct

*I **Madhavan. S (192425113)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **Madhavan. S**

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

Annexure A – Case Study

CASE STUDY 1: Smart Mobile Keyboard Prediction System

A smartphone company is developing an AI-powered keyboard application that predicts the next word while users type emails, messages, and social media posts. The language model is trained on the corpus:

"Data science is powerful, data science drives innovation, data science is evolving."

The development team employs Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis to improve prediction accuracy and handle unseen word sequences.

The following statistics are obtained from the corpus:

- C(data) = 3
- C(data science) = 3
- C(science is) = 2
- C(science drives) = 1
- C(science drives innovation) = 1

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word "science" are:

- P(is) = 0.66
- P(drives) = 0.33

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability P(science | data). Interpret how this probability influences next-word prediction in a mobile keyboard application.
2. A user types the unseen sequence "data science improves". Apply the Backoff Model and estimate the probability using lower-order n-grams. Explain why backoff is essential for real-time predictive text systems.
3. Using Deleted Interpolation, calculate the probability of the sequence "data science is". Discuss how interpolation balances reliability and coverage in sparse datasets.
4. Calculate the Entropy of the distribution P(is)=0.66 and P(drives)=0.33. Interpret the result and evaluate how entropy helps measure uncertainty and prediction confidence in intelligent keyboard systems.

CASE STUDY 2: AI-Powered Customer Support Chatbot

A multinational airline deploys a customer support chatbot that processes user messages and automatically identifies the grammatical role of words before generating responses.

Consider the following user inputs:

1. "Book a flight ticket now."
2. "This book is interesting."

The chatbot uses the Penn Treebank POS Tagset and a Hidden Markov Model (HMM) for probabilistic tagging.

Given:

- P(book | VB) = 0.6
- P(book | NN) = 0.4
- P(Start → VB) = 0.5
- P(Start → NN) = 0.5

The system must determine whether the word "book" functions as a Verb (VB) or Noun (NN) depending on context.

Questions

1. Assign appropriate Penn Treebank POS tags to every word in both sentences. Justify the tag assigned to the word "book" using contextual clues.
2. Using the HMM probabilities, compute the likelihood of "book" being tagged as a Verb (VB) in the sentence "Book a flight ticket now." Explain why the probabilistic model favors this tag.
3. Compare Rule-Based Tagging and Stochastic (HMM) Tagging in resolving lexical ambiguity. Recommend which approach is more suitable for large-scale chatbot deployment and justify your answer.
4. Evaluate the role of standardized POS tagsets and word classes in improving intent detection, response generation, and overall chatbot performance.

CASE STUDY 3: News Analytics and POS Tag Correction System

A digital news analytics company processes millions of news articles daily. The platform uses a Transformation-Based Tagger (Brill Tagger) to improve tagging accuracy after an initial POS tagging phase.

The sentence processed by the system is:

"Economic growth increases employment."

Initial POS Tags:

- economic/JJ
- growth/NN
- increases/NNS
- employment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and uncertainty evaluation.

Questions

1. Apply the transformation rule and update the POS tag sequence. Explain why the correction is linguistically appropriate.
2. Analyze the initial tagging errors and justify the corrected tag assignments using grammatical structure and sentence semantics.
3. Assuming the corpus contains the words:
    - economic (120 occurrences)
    - growth (450 occurrences)
    - increases (210 occurrences)
    - employment (380 occurrences)

    Compute the word frequency distribution and explain how frequency information supports probabilistic POS tagging.

4. Evaluate how Transformation-Based Tagging improves tagging accuracy compared with the initial tagging stage. Discuss how entropy can be used to measure uncertainty in tag assignments before and after rule application.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CASE STUDY 1: Smart Mobile Keyboard Prediction System

Understanding of the Case

A smartphone company is developing an AI-powered keyboard that predicts the next word while users type messages, emails, and social media posts. The system uses statistical language models trained on the corpus:

"Data science is powerful, data science drives innovation, data science is evolving."

The language model applies:

- Bigram Language Model
- Trigram Language Model
- Backoff Model
- Deleted Interpolation
- Entropy Analysis

These techniques improve prediction accuracy, especially when users type incomplete or unseen word sequences.

Question 1 — Compute the MLE of the Bigram Probability P(science | data)

Formula

P(science | data) = C(data science) / C(data)

Given

- C(data) = 3
- C(data science) = 3

Calculation

P(science | data) = 3 / 3 = 1.0

P(science | data) = 1.0

Interpretation

The probability of "science" occurring after "data" is 100% in the training corpus. This means:

- Whenever the user types "data", the keyboard confidently predicts "science" as the next word.
- Since the probability is the highest possible value (1.0), this prediction receives the highest priority.
- High-probability predictions reduce typing effort and improve typing speed.

Application in Mobile Keyboard

- Faster text entry
- Better prediction accuracy
- Reduced typing errors
- Improved user experience

Question 2 — Apply the Backoff Model to the unseen sequence "data science improves"

Step 1 — Trigram

The trigram "data science improves" does not appear in the corpus. Therefore, P(improves | data science) = 0. Backoff moves to the Bigram Model.

Step 2 — Bigram

The bigram "science improves" also does not appear. P(improves | science) = 0. Backoff moves to the Unigram Model.

Step 3 — Unigram

The word "improves" is not present in the corpus. Hence, P(improves) = 0.

P(data science improves) ≈ 0

If smoothing is applied in practical systems, a very small probability (instead of zero) is assigned to unseen words.

Why Backoff is Important

Backoff solves the data sparsity problem. Benefits include:

- Handles unseen word combinations.
- Prevents prediction probability from becoming exactly zero.
- Improves robustness of keyboard prediction.
- Provides reasonable predictions even for new sentences.
- Makes real-time typing smoother and more reliable.

Application in Smart Keyboard

Users frequently type new words or uncommon phrases. Without Backoff, prediction fails. With Backoff, the keyboard falls back to simpler language models and still generates useful suggestions.

Question 3 — Deleted Interpolation Probability of "data science is"

Formula

$P = \lambda_1 P\_tri + \lambda_2 P\_bi + \lambda_3 P\_uni$

Given Weights

- $\lambda_1 = 0.5$
- $\lambda_2 = 0.3$
- $\lambda_3 = 0.2$

Step 1 — Trigram Probability

P(is | data science) = C(data science is) / C(data science)

The phrase "data science is" appears 2 times.

P_tri = 2 / 3 = 0.667

Step 2 — Bigram Probability

P(is | science) = C(science is) / C(science)

“Science” appears 3 times; “science is” appears 2 times.

$P\_bi = 2 / 3 = 0.667$

Step 3 — Unigram Probability

Total words in corpus: Sentence 1 (Data Science Is Powerful) = 4, Sentence 2 (Data Science Drives Innovation) = 4, Sentence 3 (Data Science Is Evolving) = 4. Total words = 12. The word “is” appears 2 times.

$P\_uni = 2 / 12 = 0.167$

Step 4 — Deleted Interpolation

$P = 0.5(0.667) + 0.3(0.667) + 0.2(0.167)$

$= 0.3335 + 0.2001 + 0.0334 = 0.567$

P(data science is) = 0.567

Discussion

Deleted Interpolation combines trigram, bigram, and unigram information instead of depending on only one model. Advantages:

- Reduces data sparsity.
- Improves prediction accuracy.
- Gives reliable probabilities.
- Handles unseen contexts effectively.
- Provides better coverage for real-world typing.

This makes predictive keyboards more accurate and dependable.

Question 4 — Calculate the Entropy

Given P(is) = 0.66 and P(drives) = 0.33.

Formula

$H = -\Sigma P(x) \log_2 P(x)$

Step 1 — For “is”

$- 0.66 \log_2(0.66) = 0.396$

Step 2 — For “drives”

$- 0.33 \log_2(0.33) = 0.528$

Step 3 — Total Entropy

$H = 0.396 + 0.528 = 0.924$ bits

H ≈ 0.92 bits

Interpretation

An entropy of 0.92 bits indicates moderate uncertainty.

- The model strongly favors “is”.
- There is still some chance of predicting “drives”.
- Lower entropy means the prediction is more confident.

Role of Entropy in Intelligent Keyboard Systems

Entropy measures prediction uncertainty. Benefits include:

- Evaluates prediction confidence.
- Helps rank word suggestions.
- Improves recommendation quality.
- Detects uncertain contexts.
- Enhances overall typing experience.

For this corpus, the relatively low entropy indicates that the keyboard can confidently predict “is” after “science”, leading to faster and more accurate text suggestions.

Overall Conclusion — Case Study 1

The Smart Mobile Keyboard Prediction System uses statistical language models to generate accurate next-word predictions. The Bigram MLE assigns a probability of 1.0 to “science” following “data”, demonstrating a perfect prediction within the training corpus. When unseen sequences occur, the Backoff Model ensures the system remains functional by reverting to lower-order models. Deleted Interpolation combines trigram, bigram, and unigram probabilities to balance accuracy with robustness, producing an interpolated probability of 0.567 for the sequence “data science is”. Finally, the entropy value of 0.92 bits indicates moderate uncertainty, showing that the model predicts “is” with high confidence while still considering alternative words. Together, these techniques enable intelligent, reliable, and efficient real-time predictive text in modern mobile keyboard applications.

CASE STUDY 2: AI-Powered Customer Support Chatbot

Understanding of the Case

A multinational airline deploys a customer support chatbot that processes user messages and automatically identifies the grammatical role of words before generating responses. The chatbot must correctly interpret words whose part of speech changes with context — for example, "book" can be a verb (to reserve) or a noun (a physical/reading object).

Two representative user inputs are considered:

- "Book a flight ticket now."
- "This book is interesting."

The chatbot resolves this ambiguity using the Penn Treebank POS Tagset together with a Hidden Markov Model (HMM) for probabilistic tagging. Given:

- P(book | VB) = 0.6
- P(book | NN) = 0.4
- P(Start → VB) = 0.5
- P(Start → NN) = 0.5

Question 1 — Assign Penn Treebank POS Tags and Justify the Tag for "book"

Sentence 1: "Book a flight ticket now."

Book/VB a/DT flight/NN ticket/NN now/RB ./.

Sentence 2: "This book is interesting."

This/DT book/NN is/VBZ interesting/JJ ./.

Justification for "book"

In Sentence 1, "Book" occurs at the start of the sentence in an imperative construction (a command to reserve something) and is immediately followed by the determiner-noun phrase "a flight ticket", which is its grammatical object. This context identifies it as a Verb (VB).

In Sentence 2, "book" is preceded by the determiner "This" and followed by the verb "is". A noun phrase position (Determiner + Noun) confirms that "book" here functions as a Noun (NN), referring to a physical object being described.

Question 2 — HMM Likelihood of "book" as Verb (VB) in "Book a flight ticket now."

Formula

Since "book" is the first word of the sentence, its joint likelihood for a candidate tag combines the initial transition probability and the emission probability:

$$\text{Score}(tag) = P(\text{Start} \rightarrow tag) \times P(\text{book} \mid tag)$$

Calculation

$$\text{Score}(VB) = P(\text{Start} \rightarrow VB) \times P(\text{book} \mid VB) = 0.5 \times 0.6 = 0.30$$

$$\text{Score}(NN) = P(\text{Start} \rightarrow NN) \times P(\text{book} \mid NN) = 0.5 \times 0.4 = 0.20$$

Normalized Probability

P(VB | book) = 0.30 / (0.30 + 0.20) = 0.6

P(book = VB) = 0.6 > P(book = NN) = 0.4

Explanation

Both tags share the same prior transition probability from Start (0.5 each), so the decision is driven entirely by the emission probabilities. Because P(book | VB) = 0.6 is higher than P(book | NN) = 0.4, the joint (transition × emission) score for VB is higher. The HMM therefore favors tagging "book" as a Verb in this sentence, which also matches the imperative, command-style context of "Book a flight ticket now."

Question 3 — Rule-Based Tagging vs. Stochastic (HMM) Tagging

- Rule-Based Tagging: relies on hand-crafted linguistic rules and context patterns; highly interpretable and precise for cases the rules anticipate, but brittle — every new ambiguity or domain expression requires a new rule, and maintenance does not scale.
- Stochastic (HMM) Tagging: learns transition and emission probabilities from annotated corpora; generalizes automatically to new sentences and word usages, and improves as more data becomes available, but needs sufficient training data and is less transparent than explicit rules.

Recommendation

For large-scale chatbot deployment, Stochastic (HMM) Tagging is the more suitable approach. Airline customer queries are highly variable in phrasing, and a rule-based system would require an ever-growing, hard-to-maintain rule set to keep pace. An HMM-based (or broader statistical/neural) tagger scales naturally with data, adapts to new expressions, and handles ambiguity — such as "book" as VB vs. NN — probabilistically rather than through exhaustive manual rules. In practice, a hybrid design (statistical tagging refined by a small set of correction rules, as in Case Study 3) offers the best balance of scalability and precision.

Question 4 — Role of Standardized POS Tagsets and Word Classes

Standardized tagsets such as the Penn Treebank tagset, combined with reliable word-class identification, materially improve chatbot performance:

- Provide a consistent linguistic representation shared across tagging, parsing, and intent-detection modules, enabling interoperability between components.
- Resolve lexical ambiguity (e.g., "book" as verb vs. noun), which is essential for correctly identifying what the user wants to do (intent) rather than just which words were used.
- Feed downstream syntactic parsing and semantic role labeling, which underlie accurate intent classification and slot filling (e.g., recognizing "book" + "flight" as a booking action with a travel object).
- Support more natural, grammatically correct response generation, since the system understands sentence roles rather than treating text as an unordered set of words.
- Improve overall reliability and reduce misclassification errors, leading to faster resolution and a better customer experience.

Overall Conclusion — Case Study 2

The AI-Powered Customer Support Chatbot demonstrates how Penn Treebank POS tagging and Hidden Markov Model probabilities resolve lexical ambiguity in real user queries. By comparing transition and

emission probabilities, the system correctly identifies “book” as a Verb (VB) in an imperative booking request with a likelihood of 0.6, versus a Noun (NN) when preceded by a determiner. While rule-based tagging offers precision for known patterns, stochastic HMM tagging offers the scalability needed for large, varied volumes of customer messages. Together, standardized tagsets and probabilistic word-class identification allow the chatbot to understand user intent accurately and generate coherent, context-appropriate responses.

CASE STUDY 3: News Analytics and POS Tag Correction System

Understanding of the Case

A digital news analytics company processes millions of news articles daily. The platform uses a Transformation-Based Tagger (Brill Tagger) to improve tagging accuracy after an initial POS tagging phase, and also performs corpus-based frequency analysis and uncertainty evaluation.

The sentence processed by the system is:

"Economic growth increases employment."

Initial POS Tags:

- economic/JJ
- growth/NN
- increases/NNS
- employment/NN

A learned transformation rule states: “Change NNS to VBZ if the preceding word is tagged as NN.”

Question 1 — Apply the Transformation Rule

Step 1

Check the word immediately preceding “increases”: it is “growth”, tagged NN. Since the condition (“preceding word is NN”) is satisfied, the rule fires and NNS is changed to VBZ.

Updated Tag Sequence

economic/JJ growth/NN increases/VBZ employment/NN

Why the Correction is Linguistically Appropriate

The sentence follows a Subject–Verb–Object structure: “growth” (subject, NN) performs the action “increases” (verb) on “employment” (object, NN). A well-formed English sentence requires a finite verb; tagging “increases” as NNS (plural noun) would leave the sentence without a verb and make the parse ungrammatical. Re-tagging it as VBZ (present tense, third-person singular verb) correctly reflects both its syntactic position and its semantic role of describing an action performed by the subject.

Question 2 — Analysis of Initial Tagging Errors

The initial tagger appears to have relied on a surface-level, morphology-based heuristic: any word ending in “-s” is tagged as a plural noun (NNS), regardless of context. This ignores the surrounding syntactic structure.

- Grammatical justification: the sentence needs exactly one finite verb; “increases” is the only candidate, so it must be VBZ, not NNS.
- Positional evidence: it directly follows a noun (“growth”) and directly precedes another noun (“employment”) — the classic Noun–Verb–Noun pattern of a transitive sentence.
- Semantic evidence: “increases” denotes a change of state caused by “growth” and affecting “employment”, which is an action/relation, not an entity — consistent with verb semantics rather than noun semantics.

These grammatical and semantic cues justify overriding the initial NNS tag with the corrected VBZ tag.

Question 3 — Word Frequency Distribution

Given corpus counts: economic = 120, growth = 450, increases = 210, employment = 380.

Total words = 120 + 450 + 210 + 380 = 1160

Relative Frequencies

P(economic) = 120 / 1160 = 0.103 (10.3%)

P(growth) = 450 / 1160 = 0.388 (38.8%)

P(increases) = 210 / 1160 = 0.181 (18.1%)

P(employment) = 380 / 1160 = 0.328 (32.8%)

How Frequency Supports Probabilistic POS Tagging

Corpus word-frequency counts form the basis of the unigram and emission probabilities used in statistical taggers such as HMMs. Words that occur more frequently in the training corpus (e.g., "growth" at 38.8%) yield more statistically reliable probability estimates, reducing the variance of the tagger's predictions. These frequency-derived probabilities act as priors that the tagger combines with contextual (transition) probabilities to choose the most likely tag for each word, which is precisely the mechanism that allows transformation-based and stochastic taggers to make confident, data-driven decisions rather than relying on fixed rules alone.

Question 4 — Transformation-Based Tagging vs. Initial Tagging, and the Role of Entropy

Accuracy Improvement

The initial tagging stage assigns tags using simple, context-free heuristics (e.g., surface morphology), which produces systematic errors such as tagging a verb ending in "-s" as a plural noun. Transformation-Based Tagging (the Brill Tagger) starts from this baseline and then applies an ordered set of learned, context-sensitive rules — such as "change NNS to VBZ if the preceding word is NN" — that specifically target and correct these recurring error patterns. Because each rule is learned from labeled training data to maximize accuracy, successive passes progressively reduce the error rate while remaining interpretable, unlike a purely statistical black-box model.

Entropy as a Measure of Tagging Uncertainty

Entropy quantifies how uncertain the tagger is about a word's tag. For "increases" before correction, suppose the tagger considered NNS and VBZ roughly equally likely:

P(NNS) = 0.5, P(VBZ) = 0.5

$H_{before} = -[0.5 \log_2(0.5) + 0.5 \log_2(0.5)] = 1$ bit

This 1-bit entropy reflects maximum uncertainty between the two candidate tags. After the transformation rule applies and confidently resolves the tag to VBZ:

P(VBZ) = 1.0, P(NNS) = 0

$H_{after} = -[1.0 \log_2(1.0)] = 0$ bits

Entropy drops from 1 bit (before) to 0 bits (after)

This reduction in entropy demonstrates, quantitatively, that Transformation-Based Tagging increases the tagger's confidence and removes ambiguity, directly corresponding to the improvement in tagging accuracy.

Overall Conclusion — Case Study 3

The News Analytics and POS Tag Correction System illustrates how Transformation-Based Tagging refines an initial, error-prone tagging pass into a linguistically accurate one. Applying the rule “change NNS to VBZ if preceded by NN” correctly re-tags “increases” as a verb, restoring the sentence's Subject–Verb–Object structure. Corpus frequency statistics (ranging from 10.3% for “economic” to 38.8% for “growth”) provide the probabilistic foundation used by statistical taggers, while entropy analysis — dropping from 1 bit to 0 bits after correction — offers a quantitative measure of how transformation rules reduce tagging uncertainty and improve overall accuracy for high-volume news article processing.